Semi-supervised learning

$P(x)$ contains some information about $P(y|x)$
↳ we can get additional info from unlabelled data

Assumptions

1. Smoothness: two data points close together ⇒ same label
2. Low density: boundary should not pass through areas of high density
3. Manifold: data on same low-D manifold ⇒ same label
4. Clustering: data in same cluster ⇒ same label
   ↳ some mapping func we dont have access to.

Inductive vs Transductive

INDUCTIVE

- Trains model and labels unlabelled points never encountered
- Trains predictive model
- If new data: predict
- predict any new point
- less computation
- types → preprocessing
  → objective function
  → pseudo-labelling

TRANSDUCTIVE

- Trains model and labels unlabelled points already encountered.
- No predictive model
- If new data: rerun algorithm
- predicts only test set
- More computation.
- no training, testing, input space: info propagated by direct conn b/w points
- usually graph based
  LPA → affinity graph

graph construction → graph (weighted: extent of similarity)
node = data point
edge = pairwise similarity
↓
inference
assign labels

Intrinsically semi-supervised:

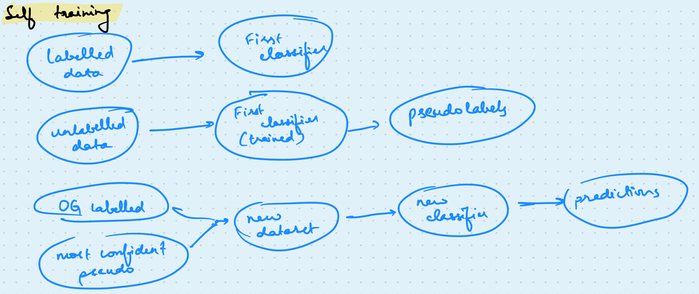
Account for unlabelled data in objective function

Unsupervised preprocessing:

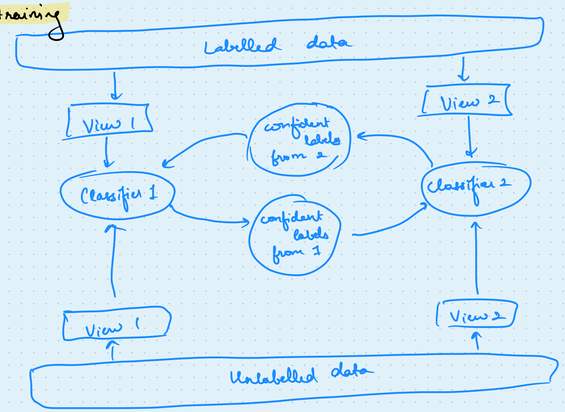
Augment labelled data w/ info from unlabelled before passing to predictive model
Eg: PCA, VAE

Wrapper methods

Self training



Co-training



Views:
separate, independent subset of attr. that is sufficient to decide class

Process

1. self training
2. confident pred select
3. co-training → augment each other
4. retrain

} Repeat until convergence

Assumptions

Consensus: more similar learner pred, better artificial datapoint
Complementarity: learners can learn from each other
Compatibility: preds from diff model → same prob. dist.
Conditional Independence: restrictive, usually relaxed
Sufficiency: each regressor is enough to build a model (eg: no of attributes)

self training ⊂ co-training

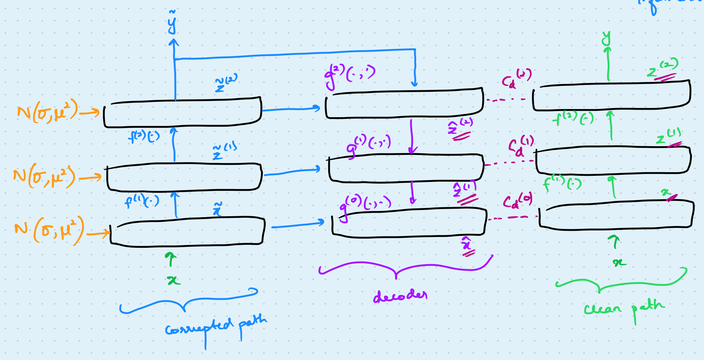
Deep SSL: Ladder Network

global supervised target
local unsupervised target
} optimises for both simultaneously.

Supervised: trained on labelled data, typical feedforward

Unsupervised:

- Based off of denoising autoencoder
- Decoder with skip connections to every encoder layer
  ↳ reconstructs og input from encoder output
  ↳ if not match, penalise
  ↳ forces encoder to learn features that preserve information



1) Two forward passes through the encoder B: $f^{(i)}$ here

1. **Clean pass**: feed the clean input x through the encoder to compute all clean pre-activations $z^{(l)}$ and activations $h^{(l)}$. Apply batch normalization to each pre-activation before the nonlinearity. These clean activations act as targets for denoising.
2. **Corrupted pass**: feed a noisy version $\tilde{x} = x + \mathcal{N}(0, \sigma^2)$ into the encoder. At every layer, add Gaussian noise to the pre-activation (or to the activation depending on implementation) and apply batch normalization to get $\tilde{z}^{(l)}$ and $\tilde{h}^{(l)}$. The corrupted encoder will also be used for supervised prediction.

Why two passes
You need the clean $z^{(l)}$ (targets) and the corrupted $\tilde{z}^{(l)}$ (inputs for denoising) during the same minibatch.

2) Supervised head uses the corrupted encoder

3. Compute the network's prediction $\tilde{y}$ from the top layer of the **corrupted** encoder. The supervised loss L_s compares $\tilde{y}$ to the true label y. Using the corrupted representation for prediction forces the encoder to learn robust features.

3) Decoder reconstructs clean pre-activations from top down

4. Start the decoder at the topmost layer. The decoder aims to recreate each clean pre-activation $z^{(l)}$, producing $\hat{z}^{(l)}$.
5. At layer l the decoder's denoising function uses two inputs: the upward reconstruction from layer $l+1$ (context from higher levels) and the corrupted $\tilde{z}^{(l)}$ from the corrupted encoder. Formally,

$$\hat{z}^{(l)} = g^{(l)}(\tilde{z}^{(l)},\ \hat{z}^{(l+1)})$$

so the decoder performs **lateral** denoising (uses the local corrupted signal) and **vertical** guidance (uses higher-level, cleaner structure). ↓

Intuition
Higher-level reconstructions tell the layer what global, semantically meaningful patterns should look like; the corrupted local signal encodes noisy detail. g combines both to produce a cleaner estimate.

4) Layer-wise denoising costs

6. For every layer compute the squared reconstruction error:

$$C_d^{(l)} = \|\hat{z}^{(l)} - z^{(l)}\|^2$$

then scale by a hyperparameter λ_l. Shallow layers typically get larger λ_l because their reconstruction matters more for raw input structure.

5) Total loss and optimization

7. Total loss is $L = L_s + \sum_l \lambda_l C_d^{(l)}$.
8. Backpropagate this combined loss through the whole model: decoder weights, encoder weights (both passes share encoder parameters), and parameters inside each denoising function $g^{(l)}$ are updated jointly.

Important
Although we run the encoder twice, the **parameters are shared**. Gradients flow through both the corrupted path (supervised) and the denoising objectives (unsupervised), so encoder parameters learn to produce representations that are both discriminative and easy to denoise.

Batch normalisation: applied to preactivations to keep values on the same range

Γ - Network

Denoising cost only on top layer, most of decoder omitted.

